



LeetCode 1480 — Running Sum of 1D Array

1. Problem Title & Link

Title: LeetCode 1480: Running Sum of 1D Array

Link: <https://leetcode.com/problems/running-sum-of-1d-array/>

2. Problem Statement (Short Summary)

Given an array `nums`, return the running sum.

Running sum means:

`runningSum[i] =`

`nums[0] + nums[1] + ... + nums[i]`

3. Examples (Input → Output)

Example 1

Input:

`nums = [1,2,3,4]`

Output:

`[1,3,6,10]`

Explanation:

1

$1+2 = 3$

$1+2+3 = 6$

$1+2+3+4 = 10$

Example 2

Input:

`nums = [1,1,1,1,1]`

Output:

`[1,2,3,4,5]`

Example 3

Input:

`nums = [3,1,2,10,1]`

Output:

`[3,4,6,16,17]`

4. Constraints

$1 \leq \text{nums.length} \leq 1000$



$-10^6 \leq \text{nums}[i] \leq 10^6$

5. Core Concept (Pattern / Topic)

Prefix Sum

This is usually the first Prefix Sum problem students learn.

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every index:

Calculate sum from 0 to i

Example:

$i=0 \rightarrow$ sum first 1 element

$i=1 \rightarrow$ sum first 2 elements

$i=2 \rightarrow$ sum first 3 elements

Time:

$O(n^2)$

Optimized Thinking

Instead of recalculating:

`runningSum[i]`

=

`runningSum[i-1]`

+

`nums[i]`

Reuse previous work.

Example:

`nums`

`[1,2,3,4]`

`running`



1

1+2 = 3

3+3 = 6

6+4 = 10

7. Visual / Intuition Diagram (ASCII)

```
nums
1   2   3   4

running

1

1+2=3

3+3=6

6+4=10
Output:
[1,3,6,10]
```

8. Pseudocode

```
running = 0

for each num

    running += num

    add running to answer

return answer
```

9. Code Implementation

Python

```
class Solution:
    def runningSum(self, nums):

        running = 0
```



```
result = []

for num in nums:

    running += num

    result.append(running)

return result
```

Java

```
class Solution {

    public int[] runningSum(int[] nums) {

        int[] result = new int[nums.length];

        result[0] = nums[0];

        for(int i = 1; i < nums.length; i++) {

            result[i] = result[i - 1] + nums[i];

        }

        return result;

    }

}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Single traversal.

Space Complexity

$O(n)$

Output array.

11. Common Mistakes / Edge Cases

Mistake 1

Resetting running sum inside loop.

Mistake 2

Starting from wrong index.

Mistake 3

Using nested loops.

12. Detailed Dry Run

Input:

[1,2,3,4]

Nu m	Running Sum	Result
1	1	[1]
2	3	[1,3]
3	6	[1,3,6]
4	10	[1,3,6,10]

Output:

[1,3,6,10]

13. Common Use Cases (Real-Life / Interview)

Bank balance tracking

Sales accumulation

Score aggregation

Inventory totals

14. Common Traps

Trap 1

Confusing running sum with total sum.

Trap 2

Not storing intermediate sums.

15. Builds To (Related LeetCode Problems)

LC 1480 Running Sum

↓

LC 303 Range Sum Query

↓

LC 724 Pivot Index

↓

LC 560 Subarray Sum Equals K



16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Nested Loops	$O(n^2)$	$O(n)$	Slow
Prefix Sum	$O(n)$	$O(n)$	Optimal

17. Why This Solution Works

Every running sum equals:

Previous Running Sum

+

Current Number

So each answer is built correctly in one pass.

18. Variations / Follow-Up Questions

1. Compute running product.
2. Running maximum.
3. Running average.
4. Prefix XOR.